

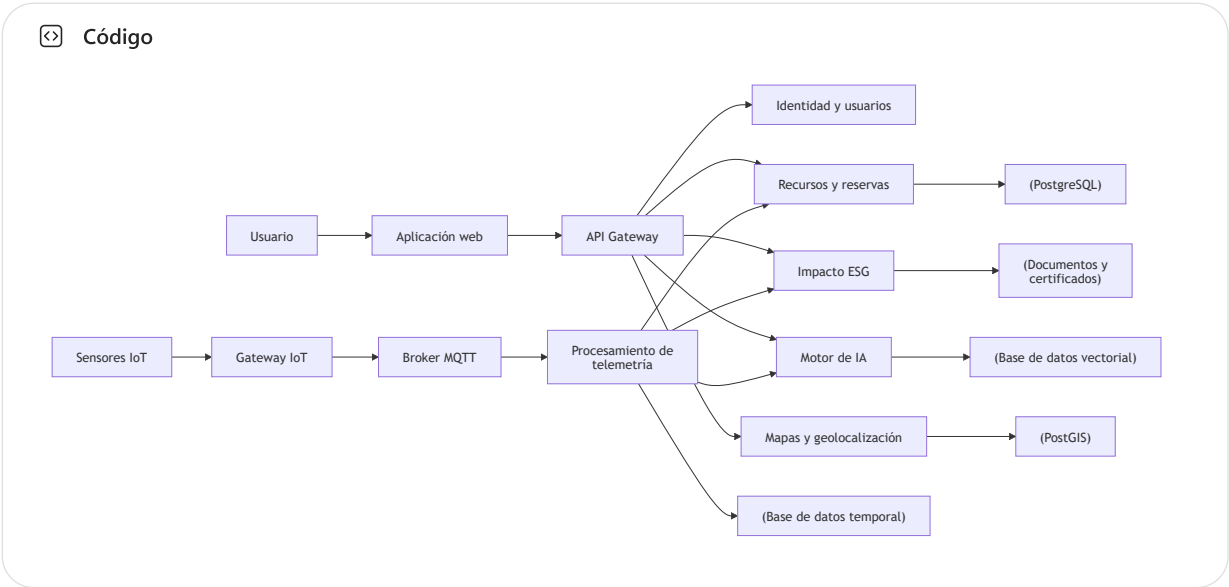
EcoSimbiosis

Arquitectura propuesta de la aplicación

La aplicación **EcoSimbiosis Hub** puede implementarse mediante una arquitectura web modular orientada a servicios. Su función es conectar organizaciones que disponen de recursos infrautilizados con otras que los necesitan, incorporando información geográfica, sensores IoT, certificación ESG y recomendaciones generadas mediante inteligencia artificial.

Los prototipos HTML representan la interfaz y el flujo funcional. Las APIs, sensores y componentes de IA descritos a continuación constituyen una propuesta técnica para convertirlos en una aplicación real.

1. Visión general



La arquitectura se divide en seis bloques:

1. Frontend web accesible
2. Backend y API Gateway
3. Servicios de negocio
4. Plataforma de sensores IoT
5. Motor de inteligencia artificial
6. Persistencia, seguridad y monitorización

2. Frontend de la aplicación

El frontend corresponde a las cuatro pantallas prototipadas:

1. Eco-Mapa interactivo
2. Ficha de recurso y certificación ESG
3. Centro de Matching e IA Predictiva
4. Dashboard de Impacto y Mercado

Podría desarrollarse como una **Single Page Application**, utilizando React, Angular o Vue. Los HTML actuales servirían como referencia visual para construir los componentes definitivos.

Responsabilidades del frontend

- Mostrar los recursos disponibles en el mapa.
- Aplicar filtros por tipo de recurso, sector, distancia y disponibilidad.
- Consultar la ficha técnica de cada activo.
- Visualizar datos procedentes de sensores.
- Solicitar y gestionar reservas.
- Mostrar recomendaciones generadas por IA.
- Consultar métricas ESG.
- Exportar informes.
- Permitir la navegación mediante teclado y tecnologías de apoyo.

El navegador no debe conectarse directamente con los sensores ni con la base de datos. Todas las operaciones deben pasar por el backend.

3. API Gateway y backend

El **API Gateway** actúa como punto de entrada único para el frontend. Puede implementarse con Spring Boot, ASP.NET Core, Node.js o FastAPI.

Su dirección podría ser:

 Plain Text

```
1 https://api.ecosimbiosis.example.eu
2
3
```

Funciones del API Gateway

- Validar el token de acceso.
- Comprobar los permisos del usuario.
- Limitar el número de solicitudes.
- Redirigir cada petición al servicio correspondiente.
- Registrar las operaciones relevantes.
- Generar identificadores de trazabilidad.
- Unificar respuestas y errores.
- Publicar la documentación OpenAPI.

Por ejemplo, el frontend podría recuperar una ficha de recurso mediante:

 HTTP

```
1 GET /api/v1/resources/RES-000145
2 Authorization: Bearer <token>
3 Accept: application/json
4
```

Respuesta simplificada:

 JSON

```
1 {
```

```
2  "id": "RES-000145",
3  "name": "Cámara climática",
4  "owner": {
5    "id": "ORG-00031",
6    "name": "Instituto de Investigaciones Agrarias"
7  },
8  "category": "laboratory_equipment",
9  "location": {
10   "latitude": 39.4752,
11   "longitude": -6.3724
12 },
13 "availability": {
14   "status": "available",
15   "source": "iot",
16   "lastUpdate": "2026-10-05T15:40:00+02:00"
17 },
18 "esg": {
19   "estimatedCo2SavingsKg": 18.5,
20   "certificateStatus": "verified"
21 }
22 }
23
```

4. Servicios del backend

Aunque inicialmente puede construirse como un **monolito modular**, la aplicación debe organizarse en módulos independientes. Esto permite convertir posteriormente los módulos con mayor carga en microservicios.

4.1. Servicio de identidad

Gestiona:

- Usuarios.
- Organizaciones.
- Roles y permisos.
- Inicio de sesión.
- Recuperación de acceso.
- Consentimientos.
- Registro de actividad.

Roles posibles

- Administrador de la plataforma.
- Administrador de organización.
- Propietario de recurso.
- Usuario solicitante.
- Auditor ESG.
- Operador técnico.
- Usuario de consulta.

La autenticación debería utilizar **OpenID Connect** y **OAuth 2.0**. Los tokens deben tener una duración limitada y los permisos deben comprobarse en el backend, no solo en la interfaz.

4.2. Servicio de recursos

Es el núcleo del catálogo de activos. Gestiona:

- Equipamiento técnico.
- Laboratorios.
- Espacios.
- Energía disponible.
- Materiales reutilizables.
- Vehículos.
- Capacidad de almacenamiento.
- Residuos valorizables.
- Ventanas de disponibilidad.

Endpoints principales:

HTTP

```
1 GET    /api/v1/resources
2 POST   /api/v1/resources
3 GET    /api/v1/resources/{resourceId}
4 PATCH  /api/v1/resources/{resourceId}
5 DELETE /api/v1/resources/{resourceId}
6
```

También puede incluir:

HTTP

```
1 GET /api/v1/resources/{resourceId}/availability
2 GET /api/v1/resources/{resourceId}/telemetry
3 GET /api/v1/resources/{resourceId}/esg
4
```

4.3. Servicio de reservas

Gestiona el uso compartido de los recursos:

HTTP

```
1 POST   /api/v1/bookings
2 GET    /api/v1/bookings/{bookingId}
3 PATCH  /api/v1/bookings/{bookingId}/accept
4 PATCH  /api/v1/bookings/{bookingId}/cancel
5 POST   /api/v1/bookings/{bookingId}/complete
6
```

Una reserva podría pasar por los estados:

Plain Text

```
1 SOLICITADA
2   ↓
3 PENDIENTE_DE_APROBACIÓN
4   ↓
5 ACEPTADA
6   ↓
7 EN_CURSO
8   ↓
9 COMPLETADA
10  ↓
11 CERTIFICADA
12
13
```

Al completar la reserva se desencadenaría el cálculo de:

- Horas reales de utilización.
 - Ingresos obtenidos.
 - Consumo energético.
 - Emisiones evitadas.
 - Valor ESG.
 - Certificado o justificante digital.
-

4.4. Servicio geográfico

Este servicio proporciona la información utilizada en el Eco-Mapa.

Funciones principales:

- Buscar recursos dentro de un radio.
- Calcular distancias.
- Agrupar marcadores.
- Filtrar por zona.
- Determinar la organización más cercana.
- Estimar la distancia logística.
- Transformar direcciones en coordenadas.

Ejemplo:



HTTP

```
1 GET /api/v1/resources/nearby?latitude=39.47&longitude=-6.37&radiusKm=15
2
```

La información geográfica puede almacenarse en **PostgreSQL con PostGIS**. Para representar el mapa se podría utilizar OpenStreetMap mediante una biblioteca como Leaflet o MapLibre.

No es recomendable enviar al proveedor cartográfico información personal o información interna de las organizaciones. El frontend debería recibir únicamente las coordenadas necesarias y, cuando proceda, una ubicación aproximada.

5. Arquitectura de sensores IoT

Los sensores se utilizan para verificar si un recurso está disponible, en funcionamiento o preparado para su uso. También permiten calcular el impacto de manera más fiable.

La normalización puede apoyarse en **OGC SensorThings API**, que proporciona un modelo abierto y geoespacial para conectar dispositivos, observaciones y aplicaciones IoT. Su parte de *Sensing* permite gestionar y recuperar observaciones y metadatos procedentes de sistemas de sensores heterogéneos.

5.1. Sensores que puede utilizar la aplicación

Sensores de ocupación

Determinan si un espacio o equipo está siendo utilizado.

- Sensores de presencia.
- Contactos de apertura.
- Barreras infrarrojas.

- Lectores de acceso.
- Sensores de vibración.

Sensores ambientales

Adecuados para laboratorios, cámaras climáticas y almacenes.

- Temperatura.
- Humedad.
- Presión.
- Calidad del aire.
- Concentración de CO₂.
- Iluminación.

Sensores energéticos

Permiten medir el consumo real del recurso.

- Potencia instantánea.
- Energía acumulada.
- Tensión.
- Intensidad.
- Factor de potencia.
- Producción renovable.

Sensores de estado del equipo

Permiten verificar si una máquina está en funcionamiento.

- Vibración.
- Revoluciones.
- Temperatura del motor.
- Horas de operación.
- Alarmas.
- Códigos de error.

Sensores de materiales

Para depósitos, almacenes o residuos aprovechables:

- Peso.
- Nivel de llenado.
- Caudal.
- Volumen.
- Identificación mediante RFID o NFC.

5.2. Flujo de los datos de sensores

 Plain Text

```
1 Sensor
2   ↓
3 Controlador o dispositivo edge
4   ↓
5 Gateway IoT
6   ↓
7 Broker MQTT
```

```
8   ↓
9   Servicio de ingestión
10  ↓
11  Validación y normalización
12  ↓
13  Base de datos temporal
14  ↓
15  Disponibilidad, IA e indicadores ESG
16
17
```

5.3. Protocolo MQTT

Los dispositivos pueden enviar telemetría mediante **MQTT 5.0**. MQTT utiliza un modelo de publicación y suscripción, es ligero y está pensado para dispositivos con recursos o ancho de banda limitados. El estándar define tres niveles de calidad de servicio para adaptar la fiabilidad de la entrega a cada operación.

Ejemplo de temas MQTT:

Plain Text

```
1  ecosimbiosis/organizations/ORG-00031/resources/RES-000145/temperature
2  ecosimbiosis/organizations/ORG-00031/resources/RES-000145/humidity
3  ecosimbiosis/organizations/ORG-00031/resources/RES-000145/occupancy
4  ecosimbiosis/organizations/ORG-00031/resources/RES-000145/energy
5  ecosimbiosis/organizations/ORG-00031/resources/RES-000145/status
6
7
```

Ejemplo de mensaje:

JSON

```
1  {
2    "deviceId": "DEV-00214",
3    "resourceId": "RES-000145",
4    "measurement": "temperature",
5    "value": 22.4,
6    "unit": "Cel",
7    "timestamp": "2026-10-05T15:41:18+02:00",
8    "quality": "valid"
9  }
10
```

5.4. Procesamiento en el edge

No todos los datos tienen que enviarse continuamente a la nube. El gateway local puede:

- Agregar lecturas por minuto.
- Eliminar valores duplicados.
- Detectar errores evidentes.
- Guardar temporalmente los datos si falla la conexión.
- Enviar únicamente cambios significativos.
- Ocultar identificadores que no sean necesarios.
- Generar eventos de disponibilidad.

Ejemplo:

Plain Text

```
1 Temperatura cada segundo
2 reduce tráfico, almacenamiento y exposición de información.
3 Promedio local cada minuto
4 ↓
5 Solo se envía el promedio y las alertas
6
```

6. API de sensores

La aplicación puede exponer una API propia basada en el modelo de SensorThings.

HTTP

```
1 GET /api/v1/iot/devices/{deviceId}
2 GET /api/v1/iot/resources/{resourceId}/observations
3 GET /api/v1/iot/resources/{resourceId}/latest
4 GET /api/v1/iot/resources/{resourceId}/availability
5 ``
6
```

Ejemplo:

HTTP

```
1 GET /api/v1/iot/resources/RES-000145/latest
2 ``
3
```

Respuesta:

JSON

```
1 {
2   "resourceId": "RES-000145",
3   "observations": [
4     {
5       "property": "temperature",
6       "value": 22.4,
7       "unit": "Cel",
8       "observedAt": "2026-10-05T15:41:18+02:00"
9     },
10    {
11      "property": "occupancy",
12      "value": false,
13      "observedAt": "2026-10-05T15:41:20+02:00"
14    }
15  ],
16  "availability": {
17    "status": "available",
18    "confidence": 0.98
19  }
20 }
21
22
```

La disponibilidad no debería depender de una única lectura. Puede calcularse combinando:

Plain Text

```
1 Estado del equipo
2 + Ausencia de ocupación
3 + Agenda sin reservas
4 + Ausencia de alarmas
5 + Confirmación del propietario
6 = Disponibilidad operativa
7
8
```

7. Motor de inteligencia artificial

La IA se utiliza principalmente en la **pantalla de matching predictivo**, aunque también puede ayudar a detectar oportunidades y generar explicaciones.

La IA no debería controlar directamente los equipos ni aprobar automáticamente una operación económica. Su función debe ser recomendar, ordenar y explicar posibilidades, dejando la decisión final al usuario.

7.1. Comprensión de necesidades

Cuando un usuario escribe:

“Necesito una cámara climática para probar sensores agrícolas durante el fin de semana.”

El sistema extrae:

 JSON

```
1 {  
2   "resourceType": "climatic_chamber",  
3   "purpose": "sensor_testing",  
4   "sector": "agritech",  
5   "requiredPeriod": {  
6     "type": "weekend"  
7   },  
8   "requiredCapabilities": [  
9     "temperature_control",  
10    "humidity_control"  
11  ]  
12 }  
13  
14
```

Esta extracción puede realizarse mediante:

- Un modelo de procesamiento del lenguaje.
- Un modelo generativo con salida JSON estructurada.
- Un clasificador entrenado con categorías del catálogo.
- Reglas de negocio para validar el resultado.

7.2. Embeddings y búsqueda semántica

Las descripciones de necesidades y recursos se transforman en vectores numéricos llamados **embeddings**. La distancia entre esos vectores permite estimar la relación semántica entre dos textos. Los embeddings se utilizan frecuentemente en búsquedas, recomendaciones, agrupación, clasificación y detección de anomalías.

Ejemplo conceptual:

 Plain Text

```
1 Necesidad del usuario  
2   ↓  
3 Modelo de embeddings  
4   ↓  
5 Vector de la necesidad  
6   ↓  
7 Búsqueda en la base vectorial  
8   ↓  
9 Recursos semánticamente similares  
10
```

Cada recurso tendría un texto normalizado:

`<>` Plain Text

```
1 Cámara climática para ensayos de componentes electrónicos y agrícolas.
2 Control de temperatura entre -20 °C y 80 °C.
3 Control programable de humedad.
4 Disponible durante fines de semana.
5 ``
6
```

Este texto se transforma en un embedding y se almacena en una base vectorial, como:

- PostgreSQL con `pgvector`.
- Azure AI Search.
- OpenSearch.
- Qdrant.
- Weaviate.

7.3. Puntuación de matching

La puntuación no debe proceder exclusivamente del modelo generativo. Conviene aplicar una fórmula verificable:

`<>` Plain Text

```
1 Puntuación =
2   40% similitud semántica
3 + 25% disponibilidad real
4 + 20% proximidad geográfica
5 + 15% impacto ambiental
6
7
```

Ejemplo:

`<>` Plain Text

```
1 Similitud semántica:      0,96 × 40 = 38,40
2 Disponibilidad IoT:      1,00 × 25 = 25,00
3 Proximidad:              0,82 × 20 = 16,40
4 Impacto ambiental:      0,94 × 15 = 14,10
5
6 Resultado final:          -----
7                           93,90%
8
```

El resultado se muestra como:

`<>` Plain Text

```
1 Coincidencia: 94 %
2
3
```

Esta combinación mejora la explicabilidad porque cada factor puede presentarse por separado.

7.4. IA predictiva

La aplicación puede predecir:

- Próximas ventanas de inactividad.
- Demanda futura de determinados equipos.
- Probabilidad de cancelación.
- Recursos con baja ocupación.
- Posibles anomalías de consumo.
- Ingresos estimados.
- Impacto ambiental esperado.

Para ello se emplean series históricas:

 Plain Text

```
1 Reservas anteriores
2 + lecturas de sensores
3 + calendario
4 + estacionalidad
5 + tipo de organización
6 + mantenimiento programado
7 = predicción
8
9
```

Por ejemplo, si una cámara climática suele estar inactiva los sábados durante seis horas, el sistema puede sugerir su publicación. Sin embargo, el responsable de la organización debe confirmar la disponibilidad.

7.5. Explicaciones de la IA

Cada recomendación debe incluir una explicación comprensible:

 Plain Text

```
1 Esta alianza se recomienda porque:
2
3 • El recurso incorpora control de temperatura y humedad.
4 • Está disponible durante el periodo solicitado.
5 • Se encuentra a 4,8 km.
6 • Su reutilización podría evitar 18,5 kg de CO2.
7 • Los datos de disponibilidad fueron actualizados por IoT hace 3 minutos.
8
```

La explicación debería generarse a partir de datos estructurados y no únicamente de texto libre. De esta manera se reduce la posibilidad de que la IA invente información.

7.6. Supervisión humana

El flujo recomendado es:

 Plain Text

```
1 IA propone
2   ↓
3 Usuario revisa la explicación
4   ↓
5 Solicitante acepta la propuesta
6   ↓
7 Propietario valida la disponibilidad
8   ↓
9 La plataforma crea la reserva
10
```

La IA no debe:

- Confirmar reservas por sí sola.
- Modificar precios sin autorización.
- Controlar equipos críticos.
- Inventar una lectura de sensor.
- Emitir un certificado si faltan datos.
- Utilizar categorías personales sensibles.
- Ocultar por qué se ha generado una recomendación.

El Reglamento de IA de la UE establece un marco orientado a una IA fiable y a la protección de la salud, la seguridad y los derechos fundamentales. Por ello, deben documentarse el propósito, los riesgos, los datos empleados, la supervisión humana y el funcionamiento del sistema.

8. Servicio de cálculo ESG

El servicio ESG transforma las reservas y la telemetría en indicadores ambientales, sociales y económicos.

Datos de entrada

- Duración real de la reserva.
- Consumo energético.
- Distancia entre organizaciones.
- Medio de transporte.
- Tipo de recurso.
- Vida útil ampliada.
- Material reutilizado.
- Producción renovable.
- Factor de emisión aplicable.

Datos de salida

 JSON

```

1 {
2   "bookingId": "BOOK-00812",
3   "environmental": {
4     "co2AvoidedKg": 18.5,
5     "energyOptimizedKwh": 42.7,
6     "resourceLifeExtensionHours": 16
7   },
8   "social": {
9     "organizationsCollaborating": 2,
10    "sharedAccessHours": 8
11  },
12  "economic": {
13    "ownerIncome": 180,
14    "estimatedBuyerSavings": 420
15  },
16  "methodologyVersion": "ESG-2.1",
17  "status": "verified"
18 }
19
```

Cada resultado debe conservar:

- Fórmula aplicada.

- Versión de la metodología.
- Fuentes de datos.
- Lecturas empleadas.
- Fecha del cálculo.
- Identidad del verificador.
- Nivel de confianza.
- Historial de correcciones.

Así, el dashboard no muestra cifras sin contexto, sino indicadores auditables.

9. Bases de datos

La aplicación necesita distintos tipos de almacenamiento.

Base de datos relacional

PostgreSQL puede almacenar:

- Usuarios.
- Organizaciones.
- Recursos.
- Reservas.
- Contratos.
- Precios.
- Pagos.
- Certificados.
- Permisos.
- Auditoría.

Extensión geográfica

PostGIS permite almacenar:

- Coordenadas.
- Zonas.
- Radios de búsqueda.
- Distancias.
- Áreas de servicio.

Base temporal

Una base de series temporales almacena las lecturas IoT:

```
<> Plain Text

1 device_id
2 resource_id
3 property
4 value
5 unit
6 timestamp
7 quality
8
```

Puede utilizarse TimescaleDB, InfluxDB u otra tecnología equivalente.

Base vectorial

Almacena los embeddings utilizados por el buscador semántico y el motor de matching.

Almacenamiento de objetos

Contiene:

- Fotografías.
 - Manuales.
 - Certificados.
 - Informes ESG.
 - Documentos técnicos.
 - Evidencias de auditoría.
-

10. APIs externas

Además de las APIs internas, la aplicación puede integrarse con servicios externos.

API cartográfica

Se utiliza para:

- Mostrar mapas.
- Obtener teselas cartográficas.
- Geocodificar direcciones.
- Calcular rutas aproximadas.

API de factores de emisión

Proporciona factores para calcular emisiones:

```
<> Plain Text

1 electricidad consumida × factor de emisión
2 kilómetros recorridos × factor de transporte
3 material reutilizado × impacto evitado
4
5
```

La aplicación debe registrar qué fuente y qué versión del factor se utilizó.

API meteorológica

Puede ayudar a:

- Contextualizar consumo energético.
- Predecir producción solar.
- Planificar recursos exteriores.
- Detectar lecturas anómalas.

API de pagos

Si la plataforma permite contratar recursos, el pago debería delegarse en un proveedor especializado. La aplicación no debería almacenar números completos de tarjetas.

API de correo y notificaciones

Se utiliza para:

- Confirmar reservas.
 - Avisar de cambios.
 - Comunicar incidencias.
 - Enviar certificados.
 - Recordar la devolución o finalización de un recurso.
-

11. Seguridad y cumplimiento europeo

Seguridad de los sensores

Cada dispositivo debe tener una identidad propia:

 Plain Text

```
1 Dispositivo → certificado → gateway → broker MQTT
2 ``
3
```

Medidas recomendadas:

- TLS para las comunicaciones.
- Certificado individual por dispositivo.
- Rotación de credenciales.
- Lista de dispositivos autorizados.
- Separación de redes IoT.
- Firmware firmado.
- Registro de altas y bajas.
- Protección contra mensajes duplicados.
- Alertas por comportamiento anómalo.

MQTT admite comunicación bidireccional, niveles de calidad de servicio y mecanismos de seguridad apoyados en TLS y autenticación.

Protección de datos

La aplicación debe aplicar:

- Minimización de datos.
- Limitación de finalidad.
- Límites de conservación.
- Exactitud.
- Integridad y confidencialidad.
- Responsabilidad demostrable.

Estos principios forman parte del tratamiento de datos personales establecido por el RGPD. La Comisión Europea señala que solo deben recopilarse los datos necesarios para la finalidad indicada y que deben conservarse durante el tiempo estrictamente necesario.

Los sensores deberían registrar principalmente el estado del recurso, no información sobre personas.

Por ejemplo:

Plain Text

```
1 Correcto:
2   sala_ocupada = true
3
4 Evitar:
5   usuario_detectado = Clara
6
7
```

Accesibilidad

El frontend debe conservar las medidas incluidas en los prototipos:

- HTML semántico.
- Navegación mediante teclado.
- Foco visible.
- Lectura correcta de formularios.
- Tablas identificadas.
- Mensajes mediante `aria-live`.
- Contraste suficiente.
- Reflujo en pantallas pequeñas.
- Ampliación del texto.
- Compatibilidad con movimiento reducido.
- Alternativas textuales para gráficos.

12. Flujo completo de una operación

Un flujo real podría ser el siguiente:

1. Un sensor detecta que una cámara climática ha terminado su ciclo.
2. El gateway publica el estado mediante MQTT.
3. El servicio IoT valida la lectura.
4. El servicio de recursos compara el estado con la agenda.
5. Se genera una ventana de disponibilidad.
6. El motor de IA encuentra una necesidad compatible.
7. El sistema calcula distancia, disponibilidad e impacto potencial.
8. La pantalla 3 muestra una recomendación del 94 %.
9. El solicitante revisa la explicación.
10. El propietario acepta la solicitud.
11. El servicio de reservas bloquea la franja horaria.
12. Durante el uso, los sensores registran tiempo y energía.
13. Al finalizar, el servicio ESG calcula el impacto.
14. Se genera una evidencia o certificado.
15. La pantalla 4 actualiza ingresos, horas y CO₂ evitado.

13. Arquitectura recomendada para una primera versión

Para el primer desarrollo no sería necesario desplegar numerosos microservicios. Una opción proporcionada sería:

Plain Text

```
1 Frontend:
2   React + TypeScript + Tailwind CSS
3
4 Backend:
5   Spring Boot o ASP.NET Core
6   Arquitectura modular
7   API REST documentada con OpenAPI
8
9 Datos:
10  PostgreSQL
11  PostGIS
12  pgvector
13  TimescaleDB
14
15 IoT:
16  MQTT 5
17  Broker Mosquitto o equivalente
18  Gateway edge
19
20 IA:
21  Servicio independiente en Python
22  Embeddings
23  Modelo de ranking
24  Predicción de series temporales
25  Explicaciones basadas en datos estructurados
26
```

Esta solución mantiene un coste inicial razonable y deja preparado el sistema para separar posteriormente la ingestión IoT, el motor de IA y el cálculo ESG.

```
28 Contenedores Docker
29 Proxy inverso
30 Almacenamiento de objetos
31 Monitorización y registro centralizado
```

Resumen

La arquitectura combina tres fuentes principales:

- **Datos declarados**, introducidos por las organizaciones.
- **Datos observados**, obtenidos mediante sensores IoT.
- **Datos inferidos**, calculados por los modelos de IA.

La recomendación final no depende únicamente de la IA. Se construye combinando disponibilidad verificada, ubicación, compatibilidad, impacto ESG y reglas de negocio. Este diseño hace que la aplicación sea más explicable, auditable y adecuada para un entorno europeo.